GUÍA DE TRABAJOS PRÁCTICOS
Adquisición y Control Bidireccional de Datos con Raspberry Pi Pico 2 W en MicroPython

Asignatura: Procesamiento Adq. Y Análisis de Señales

Carrera: TEM

Placa Objetivo: ADC con RBP 2 W (RP2350)

Entorno / Lenguaje: Thonny IDE / MicroPython

1. Objetivos del Laboratorio

- Comprender y configurar la pila de red Wi-Fi en la Raspberry Pi Pico 2 W mediante MicroPython utilizando la librería nativa network.
- Implementar un servidor web HTTP liviano mediante sockets TCP en el puerto 80 para el intercambio bidireccional de datos.
- Adquirir y procesar señales físicas: lecturas analógicas de 12 bits (escaladas a 16 bits en MicroPython vía ADC GP26) y estados lógicos digitales (GP14).
- Controlar actuadores desde la interfaz web embebida actuando sobre salidas digitales (GP15).
- Analizar y comparar prácticamente las arquitecturas de red: Modo Estación (STA_IF) vs. Modo Punto de Acceso (AP_IF).

2. Marco Teórico Comparativo: Modo STA vs. Modo AP

La Raspberry Pi Pico 2 W incorpora el chip de conectividad inalámbrica CYW43439, permitiendo operar en dos modos de red principales. La elección entre un modo u otro impacta directamente en la topología de red, la seguridad y el flujo de tráfico de datos:

Parámetro / Característica	Modo Estación (STA_IF)	Modo Punto de Acceso (AP_IF)
Rol en la Red	Cliente Wi-Fi asociado a un AP o Router existente.	Servidor de Red / AP emisor autónomo.
Asignación de IP	Dinámica por DHCP del router (ej. 192.168.1.X).	Estática por defecto en el Pico 2 W (192.168.4.1).
Infraestructura	Requiere Router o Hotspot Wi-Fi externo.	Autónoma (No requiere router ni internet).
Acceso a Internet	Posible (si el router dispone de salida a WAN).	Sin salida a WAN (Red local cerrada).
Casos de Uso Típicos	Sistemas IoT integrados a redes locales o Cloud.	Configuración inicial, diagnóstico de campo o redes privadas.

3. Circuito de Prueba y Conexiones de Hardware

Para desarrollar ambas prácticas, los estudiantes montarán el siguiente circuito en protoboard, manteniendo la misma asignación de pines GPIO en ambas experiencias:

- **Entrada Analógica (ADC0 - GP26 / Pin 31):** Potenciómetro de 10 kΩ conectado entre 3.3V(OUT) (Pin 36) y GND. El terminal central se conecta al pin GP26.

- **Entrada Digital (Lógica - GP14 / Pin 19):** Pulsador normalmente abierto (NA) con resistencia pull-down interna configurada por software.
- **Salida Digital (Actuador - GP15 / Pin 20):** LED indicador con resistencia limitadora de 330 Ω conectada a GND.

> **Nota Técnica sobre ADC en MicroPython:** Aunque el hardware de la Raspberry Pi Pico 2 W tiene una resolución nativa de 12 bits (0 a 4095), la implementación estándar de MicroPython escala el resultado a un entero de 16 bits sin signo (0 a 65535) mediante la función read_u16(). La conversión a tensión se realiza como: V = (read_u16() * 3.3) / 65535.

4. Práctica 1: Servidor Web en Modo Estación (STA)

En esta práctica, la Raspberry Pi Pico 2 W se conectará a la red Wi-Fi del laboratorio, obtendrá una dirección IP mediante DHCP y pondrá en marcha un servidor HTTP para servir la interfaz web y responder a los comandos del usuario.

Procedimiento Paso a Paso:

1. Configurar las credenciales de la red Wi-Fi local en las variables del código.

2. Iniciar la interfaz de red en modo Estación (network.STA_IF) y verificar la asignación de la dirección IP a través de la consola REPL.

3. Crear y vincular un socket TCP en el puerto 80 para recibir peticiones de clientes HTTP en la red.

4. Navegar desde una PC o dispositivo móvil conectado a la misma red ingresando la dirección IP informada.

5. Accionar el botón de encendido/apagado del LED y verificar la actualización en tiempo real de las lecturas analógica y digital.

Código Fuente en MicroPython (Modo Estación - main_sta.py):

```
import network
import socket
import time
from machine import ADC, Pin

# Hardware: GP26 (ADC0), GP14 (Entrada Digital), GP15 (Salida LED)
adc = ADC(Pin(26))
btn_input = Pin(14, Pin.IN, Pin.PULL_DOWN)
led_output = Pin(15, Pin.OUT)

# Configuración Wi-Fi Modo Estación
WIFI_SSID = "RED_LABORATORIO"
WIFI_KEY = "CLAVE_LABORATORIO"

wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(WIFI_SSID, WIFI_KEY)

print("Conectando a Wi-Fi...")
timeout = 10
while not wlan.isconnected() and timeout > 0:
    time.sleep(1)
    timeout -= 1
```

```
if not wlan.isconnected():
    raise RuntimeError("Error de conexión Wi-Fi.")

ip_info = wlan.ifconfig()
print("IP Asignada:", ip_info[0])

def render_html(raw_adc, voltaje, estado_btn, estado_led):
    str_btn = "ALTO (1)" if estado_btn else "BAJO (0)"
    str_led = "ENCENDIDO" if estado_led else "APAGADO"
    return f"<html><body><h2>Pico 2 W - STA</h2><p>ADC GP26: {raw_adc} / 65535 ({voltaje:.2f}
V)</p><p>GP14: {str_btn}</p><p>LED GP15: {str_led}</p><a href=\"/led=on\">Encender LED</a> | <a
href=\"/led=off\">Apagar LED</a></body></html>"

addr = socket.getaddrinfo("0.0.0.0", 80)[-1]
server = socket.socket()
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind(addr)
server.listen(1)

while True:
    try:
        conn, client = server.accept()
        request = conn.recv(1024).decode("utf-8")
        if "/led=on" in request:
            led_output.value(1)
        elif "/led=off" in request:
            led_output.value(0)
        val_adc = adc.read_u16()
        volt = (val_adc * 3.3) / 65535
        resp = render_html(val_adc, volt, btn_input.value(), led_output.value())
        conn.send("HTTP/1.1 200 OK\r\nContent-Type: text/html\r\n\r\n")
        conn.send(resp)
        conn.close()
    except Exception as e:
        print("Error:", e)
```

5. Práctica 2: Servidor Web en Modo Punto de Acceso (AP)

En esta práctica, la Raspberry Pi Pico 2 W creará su propia red Wi-Fi independiente. Los estudiantes conectarán sus computadoras o teléfonos móviles directamente a la red de la Pico 2 W para interactuar con la interfaz web sin depender de la infraestructura del laboratorio.

Procedimiento Paso a Paso:

1. Definir el nombre de la red (SSID) y la clave WPA2 (mínimo 8 caracteres) para el Punto de Acceso.
2. Inicializar la interfaz de red en modo AP (network.AP_IF) y activar el servicio de red local.
3. Verificar en la consola REPL la dirección IP estática por defecto (típicamente 192.168.4.1).
4. Conectar una laptop o smartphone a la red Wi-Fi emitida por la Pico 2 W ingresando la clave configurada.
5. Abrir el navegador e ingresar la dirección 192.168.4.1 para gestionar las señales de entrada y salida.

Código Fuente en MicroPython (Modo Punto de Acceso - main_ap.py):

```
import network
import socket
import time
from machine import ADC, Pin

adc = ADC(Pin(26))
btn_input = Pin(14, Pin.IN, Pin.PULL_DOWN)
led_output = Pin(15, Pin.OUT)

# Configuracion Punto de Acceso (AP)
AP_SSID = "Pico2W_Lab_GrupoX"
AP_KEY = "Pico2W_Acceso2026"

ap = network.WLAN(network.AP_IF)
ap.config(essid=AP_SSID, password=AP_KEY)
ap.active(True)

while not ap.active():
    time.sleep(0.1)

ip_info = ap.ifconfig()
print("AP Activo:", AP_SSID, "IP:", ip_info[0])

def render_html_ap(raw_adc, voltaje, estado_btn, estado_led):
    str_btn = "ALTO (1)" if estado_btn else "BAJO (0)"
    str_led = "ENCENDIDO" if estado_led else "APAGADO"
    return f"<html><body><h2>Pico 2 W - AP</h2><p>SSID: {AP_SSID}</p><p>ADC GP26: {raw_adc}
({voltaje:.2f} V)</p><p>Entrada GP14: {str_btn}</p><p>Salida GP15: {str_led}</p><a
href=\"/led=on\">Activar Salida</a> | <a href=\"/led=off\">Desactivar Salida</a></body></html>"

addr = socket.getaddrinfo("0.0.0.0", 80)[-1]
server = socket.socket()
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind(addr)
server.listen(1)

while True:
    try:
        conn, client = server.accept()
        request = conn.recv(1024).decode("utf-8")
        if "/led=on" in request:
            led_output.value(1)
        elif "/led=off" in request:
            led_output.value(0)
        val_adc = adc.read_u16()
        volt = (val_adc * 3.3) / 65535
        resp = render_html_ap(val_adc, volt, btn_input.value(), led_output.value())
        conn.send("HTTP/1.1 200 OK\r\nContent-Type: text/html\r\n\r\n")
        conn.send(resp)
        conn.close()
    except Exception as e:
        print("Error AP:", e)
```

6. Cuestionario de Evaluación y Desafíos Prácticos

Preguntas de Análisis Técnico:

Q1: ¿Cuáles son las diferencias fundamentales entre una socket s.listen(1) bloqueante y un modelo asincrónico basado en uasyncio para gestionar múltiples clientes HTTP?

Q2: Analice la diferencia entre la resolución de 12 bits real del ADC del RP2350 y el entero de 16 bits entregado por read_u16(). ¿Cómo influye esto en la cuantización de la medición?

Q3: En un entorno industrial o de campo, ¿qué ventajas ofrece desplegar un modo AP con IP estática frente al modo STA con DHCP?

Q4: ¿Qué ocurre con las conexiones socket si el cliente cierra la pestaña del navegador abruptamente antes de recibir la respuesta HTTP completa? ¿Cómo maneja el código esta excepción?

Desafíos Prácticos Adicionales (Optativos / Evaluación Superior):

- **Desafío A - Actualización Automática Asincrónica (AJAX / Fetch API):** Modificar el archivo HTML para realizar peticiones periódicas mediante fetch en JavaScript, logrando que el valor analógico se actualice en pantalla cada 1 segundo sin recargar la página completa.
- **Desafío B - Control Proporcional con Salida PWM:** Implementar una barra deslizante en la interfaz web para enviar un valor de 0 a 100% y controlar la intensidad de brillo del LED mediante la clase machine.PWM en el GP15.